Sistema de Gestión Académica

Documento de decisiones de diseño

Actividad colaborativa 2

Silvana Arenas Molina

Kelly Stefania Martinez Martinez

Oscar Ricaurte Salamanca

Stefanny Muñoz Gasca

Lizeth Lopez Blanco

Facultad de Ciencias e Ingenierías

Universidad de Manizales

Programación I

Docente Natalia Marcela Castellanos Gómez

1 de noviembre de 2026

Introducción

La segunda actividad amplía el sistema construido en la primera entrega. Conservamos los modelos y la aplicación de consola y agregamos la abstracción de personas, los horarios y las excepciones académicas. El objetivo es incorporar la oferta de grupos y la matrícula sin crear otro proyecto.

Solución implementada y organización

Mantenemos la organización MVC. Los modelos representan las entidades y validan sus datos, las vistas reciben entradas y muestran mensajes y los controladores coordinan las acciones. main.py prepara los objetos, conecta las áreas y captura las excepciones académicas. Las búsquedas compartidas están en controllers/busquedas.py y las clases de excepción en exceptions/academicas.py para evitar repetirlas.

Los datos se mantienen en memoria durante la sesión. Cuando una búsqueda falla, main muestra el mensaje y permite regresar al menú principal sin reconstruir los objetos ni perder los cambios anteriores.

Abstracción de las personas

La documentación de Python explica que ABC y abstractmethod permiten definir clases abstractas que no pueden instanciarse mientras conserven métodos abstractos sin implementar (Python Software Foundation, s. f.-a). Aplicamos ese mecanismo a la representación general de una persona.

Person, en models/persona.py, hereda de la clase ABC del módulo abc. Su método realizar_actividad_principal() usa el decorador @abstractmethod. Así se impide crear una persona genérica sin establecer su rol y se exige que cada clase especializada implemente su actividad.

Student, Teacher y Administrative conservan los datos comunes y aportan su propia implementación de la actividad. El estudiante describe su actividad académica, el docente la

enseñanza en su departamento y el administrativo su trabajo según su cargo y dependencia. La información particular también se presenta mediante la sobreescritura de mostrar_informacion().

La opción 1 de Prueba de diseño para la actividad, accesible desde la opción 5 del menú principal, llama a comprobar_persona_abstracta() en main. Allí se intenta crear un objeto de Person y se captura el TypeError esperado, mostrando una explicación antes de continuar. Esta comprobación se ejecuta cuando el usuario la selecciona.

Conservación del polimorfismo

crear_personas_ejemplo() prepara una colección con dos estudiantes, dos docentes y dos administrativos. PersonaView.mostrar_personas() recorre esa colección con un único ciclo y llama a mostrar_informacion() y realizar_actividad_principal() sin comprobar el tipo de cada objeto. Python ejecuta la implementación de cada clase hija.

Las listas por área contienen referencias a los mismos objetos de la colección general. Por eso, al actualizar un estudiante desde su área, el cambio también aparece en la consulta de todas las personas.

Grupos, horarios y composición

Horario, en models/horario.py, almacena día, hora de inicio, hora de finalización y salón mediante atributos privados. Los setters verifican el día, el formato de las horas y que el salón tenga contenido. Los getters permiten consultar los datos y mostrar_horario() devuelve una descripción legible.

GrupoAcademico recibe objetos existentes de Asignatura y Teacher. Su lista de matriculados guarda objetos Student. Estas referencias permiten consultar información actualizada sin copiar los nombres de las entidades. crear_grupos_ejemplo() prepara G1 y G2 desde main y los entrega a GrupoController junto con los estudiantes disponibles.

El constructor de GrupoAcademico crea su propio Horario a partir del día, las horas y el salón recibidos. La relación es de composición: el horario forma parte de la organización del grupo. Un grupo contiene un horario y no es una especialización de Horario, por lo que no corresponde usar herencia. Asignaturas y docentes se reciben ya creados y pueden existir independientemente de un grupo.

GrupoView muestra las opciones y recoge los códigos. GrupoController selecciona los objetos y llama a matricular_estudiante(), retirar_estudiante() y mostrar_estudiantes(). Las reglas de cupos y duplicados permanecen en el modelo, para que no dependan de la interfaz de consola.

Excepciones personalizadas y búsquedas

La documentación de Python indica que las excepciones propias se definen mediante clases derivadas de Exception y pueden manejarse mediante try y except (Python Software Foundation, s. f.-b). Las cinco excepciones del proyecto heredan de Exception y construyen mensajes con los códigos involucrados. EXCEPCIONES_ACADEMICAS las reúne en una tupla que permite capturarlas con un solo except en main.

buscar_asignatura_por_codigo() y buscar_programa_por_codigo() recorren las listas y consultan get_code(). Antes de comparar limpian espacios y convierten los códigos a mayúsculas. Si no encuentran el registro, lanzan AsignaturaInexistenteError o ProgramaInexistenteError. Se eligió lanzar una excepción en lugar de devolver None para que la operación interrumpida tenga un tratamiento explícito y no se utilice después un objeto inexistente.

Los controladores de programas y asignaturas usan estas funciones tanto para buscar como para seleccionar el objeto que se actualizará. Las excepciones llegan a main, donde se muestran sus mensajes y se retoma el menú. Un código vacío genera ValueError, que también se captura.

matricular_estudiante() compara el código estudiantil con los matriculados y lanza EstudianteDuplicadoError si ya existe. Después comprueba la capacidad y lanza GrupoLlenoError si no queda cupo. Solo agrega el estudiante cuando ambas comprobaciones se cumplen. retirar_estudiante() busca por código estudiantil y lanza EstudianteNoMatriculadoError si no lo encuentra. Estas excepciones pasan por el controlador hasta main, que informa el problema y permite continuar sin reconstruir los grupos.

El método set_capacidad_maxima() rechaza valores que no sean enteros positivos, incluidos True y False. También impide reducir la capacidad por debajo de la cantidad de estudiantes ya matriculados. Estas validaciones conservan la capacidad anterior si el cambio no es válido.

Verificación

Para verificar comportamientos utilizamos unittest, que organiza casos de prueba y permite automatizar su ejecución (Python Software Foundation, s. f.-c). La suite contiene 41 pruebas ejecutables mediante python -m unittest discover -s tests -t . -v. Las pruebas con unittest comprueban la abstracción, el recorrido polimórfico, la consulta del horario y la conservación de los datos tras cambios inválidos. Las pruebas de integración ejecutan main con entradas simuladas y verifican búsquedas correctas, códigos inexistentes, códigos vacíos y continuidad después de los errores.

Las pruebas de integración también ejecutan el área de grupos desde main con entradas simuladas. Comprueban los dos grupos y sus horarios, la matrícula y el retiro correctos, los tres errores de matrícula y la continuidad posterior. Tras una excepción verifican que se conserva la matrícula anterior y que un retiro libera el cupo. Otra prueba comprueba que los grupos conservan las referencias a los objetos originales y que tienen horarios distintos. Las pruebas no se ejecutan automáticamente al abrir la aplicación.

Uso de inteligencia artificial

Durante el desarrollo utilizamos ChatGPT/Codex y Claude de Anthropic como herramientas de apoyo para comprender los requisitos y conceptos de programación orientada a objetos, organizar las tareas, revisar errores y preparar documentación. También se usó ChatGPT/Codex para generar y ajustar bloques de código de la interfaz de consola y de la integración de los componentes.

Las pruebas unitarias y de integración incorporadas en la carpeta tests/ fueron generadas con apoyo de inteligencia artificial. ChatGPT/Codex escribió las pruebas de personas, horarios, grupos y navegación por consola, y las ejecutó para comprobar los resultados. La revisión detectó los getters faltantes de Horario y las pruebas de grupos detectaron errores en las validaciones de capacidad de GrupoAcademico. Las correcciones de capacidad y sus pruebas también se elaboraron con esta herramienta. Silvana declaró el uso de Claude para escribir su código de excepciones y búsquedas y compartió las pruebas que acompañan ese aporte.

Stefanny declaró apoyo de IA para comprender el módulo abc y preparar las expresiones regulares de Horario. Kelly Stefania Martinez Martinez utilizó inteligencia artificial como apoyo para desarrollar la vista y el controlador de grupos académicos, organizar el menú de consola y comprender cómo conectar las consultas, la matrícula y el retiro con el modelo. Este apoyo incluyó la generación de los bloques de código de ambos archivos.

El uso de estas herramientas no sustituye la responsabilidad de cada integrante de revisar, comprender y poder explicar el código, las pruebas y las decisiones de diseño que incorpora a la entrega. Cada integrante es responsable de verificar que esta declaración refleje el apoyo utilizado en sus aportes.

Conclusiones

La segunda versión conserva el sistema desarrollado en la primera actividad y amplía sus responsabilidades con grupos académicos y operaciones de matrícula y retiro. Se reutilizan los objetos de asignaturas, docentes y estudiantes, de modo que cada área consulta las mismas entidades y los cambios de una sesión no producen copias independientes de sus datos.

La abstracción de Person exige que las clases especializadas definan su actividad principal y mantiene el recorrido polimórfico de la consulta general. La composición permite que cada GrupoAcademico organice su propio Horario sin confundir esta relación con una especialización por herencia. Las excepciones se generan donde se detectan los problemas y se capturan en main para informar al usuario y continuar la ejecución.

La separación entre modelo, vista y controlador facilita revisar las reglas académicas y probar la interacción de consola. Las pruebas unitarias y de integración comprobaron los casos solicitados y permitieron corregir las validaciones de capacidad. La solución mantiene los datos en memoria y no incorpora persistencia; por ello, los registros iniciales se recuperan al iniciar una nueva sesión y las actualizaciones se pierden al cerrar el programa.

Referencias

Python Software Foundation. (s. f.-a). *abc — Abstract base classes*. Python 3.12 documentation. https://docs.python.org/3.12/library/abc.html

Python Software Foundation. (s. f.-b). *Errors and exceptions*. Python 3.12 documentation. https://docs.python.org/3.12/tutorial/errors.html

Python Software Foundation. (s. f.-c). *unittest — Unit testing framework*. Python 3.12 documentation. https://docs.python.org/3.12/library/unittest.html